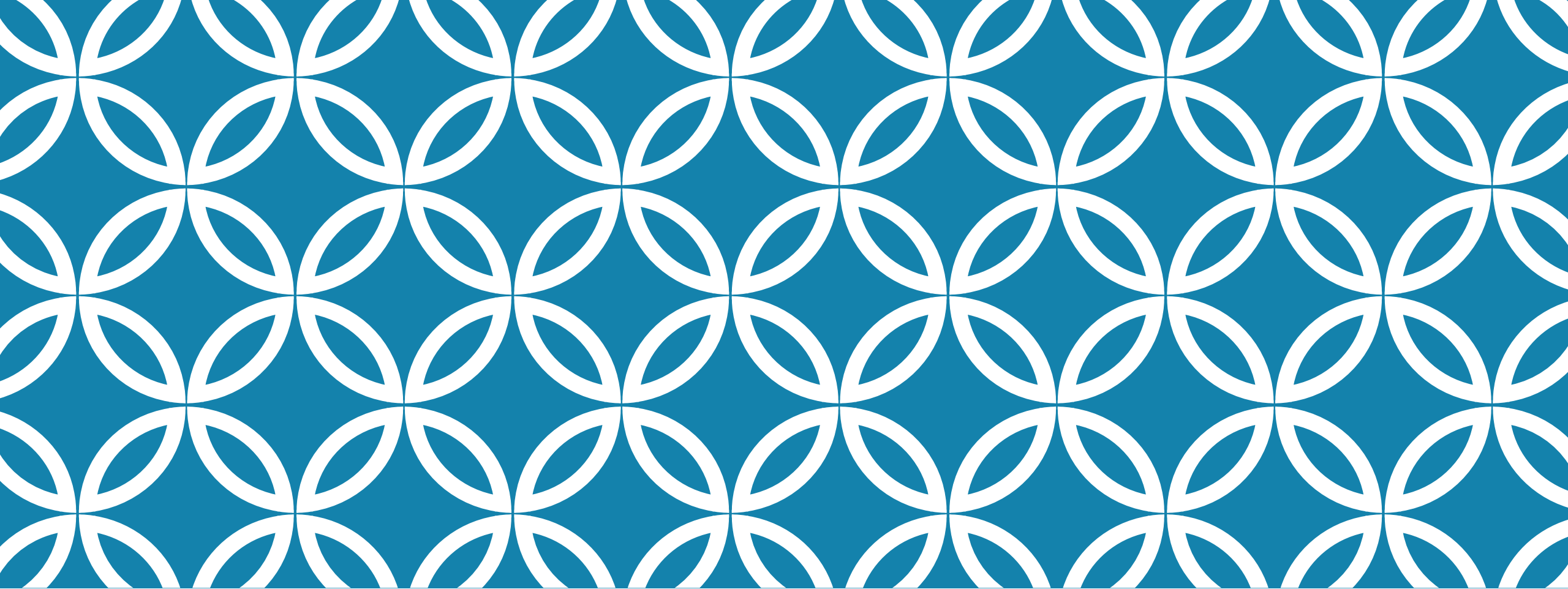




# STATIC PROGRAM ANALYSIS

Lecture 09 – Data-flow framework

# COMMON FEATURES AMONG ANALYSES

Four classic analyses:

- Live variables
- Reaching definitions
- Very busy expressions
- Available expressions

Define what data to propagate

Define how change the data for a statement

Define the direction of the flow (forward or backward)

Define how to merge flows

Use the same algorithm for solving IN/OUT equations

# MONOTONE FRAMEWORK

A directed graph  $CFG(V, E)$ , statements  $n \in V$ , source node  $s \in E$   
(no incoming edges)

Dataflows in/out of nodes

$$IN(n) = \begin{cases} \iota & \text{if } n \in E \\ \sqcup \{OUT(n') \mid n' \in F.pred(n)\} & \text{otherwise} \end{cases}$$

$$OUT(n) = f_n(IN(n))$$

where

$\sqcup$  is  $\cap$  or  $\cup$

$F$  is either  $CFG$  or  $CFG^{Reverse}$

$\iota$  the initial ( $F = CFG$ ) or final ( $F = CFG^{Reverse}$ ) analysis information

$f_n$  is the transfer function that states how flow is changed by a statement  $n$

$$F = CFG$$

- Forward analysis
- $E$  is the set of start statements (only one)

$$F = CFG^{Reverse}$$

- Backward analysis
- $E$  is the set of return/exception statements (can be many)

$\cup$

- The least sets that solve the IN/OUT equations
- Properties satisfied by at least one execution path
- Initialize IN/OUT with  $\{\}$
- May analysis

$\cap$

- The greatest sets that solve the IN/OUT equations
- Properties satisfied by all execution paths
- Initialized IN/OUT with the maximum set
- Must analysis

# ABSTRACT DOMAIN

Represent what type of data to compute

- $l \in L$
- $l$ : a set of expressions, a set of variables
- $L$ : a set of all possible set of expressions in P

Combination operator  $\sqcup$ :  $\wp(L) \rightarrow L$

- Specifies how to combine data from several paths

Elements  $l \in L$  create a lattice with a subsumption relation

- May analysis
  - $l_1 \subseteq l_2$
  - the bottom  $\perp$  element is  $\{\}$
  - the top  $\top$  element all values
- Must analysis
  - $l_1 \supseteq l_2$
  - the bottom  $\perp$  element is all values
  - the top  $\top$  element is  $\{\}$

Reaching definitions (May analysis)

- $L = \wp(\text{Var} \times \text{LineNumber})$
- On merges union flows, thus  $l$  goes “up” the lattice to all  $(\text{var}@\text{line})$  tuples

Available expressions (Must analysis)

- $L = \wp(\text{AExp})$  all set of arithmetic expression in P
- On merges intersect flow, thus  $l$  goes “up” the lattice to no values  $\{\}$

# TRANSFER FUNCTION

$f_n: L \rightarrow L$ , for a statement at line  $n$

The transfer function should be monotone

- if  $l_1 \sqsubseteq l_2$  then  $f_n(l_1) \sqsubseteq f_n(l_2)$
- Otherwise the iterative algorithm might not converge

An analysis must have a set of monotone functions  $\mathcal{F}$  over  $L$

- $\mathcal{F}$  contains all transfer functions for all lines
- $\mathcal{F}$  contains an identity function (do “nothing”)
- $\mathcal{F}$  is closed under composition – the sequencing of statements

Distributed frameworks (all finite  $L$ )

- $\forall f \in \mathcal{F}: f(l_1 \sqcup l_2) = f(l_1) \sqcup f(l_2)$

Focus on finite domains  
for now

# COMPLETE DEFINITION OF ANALYSIS

- The complete lattice,  $L$
- The space of functions,  $\mathcal{F}$
- a finite flow,  $F$
- a finite set of extreme labels  $E$
- an extremal value,  $\iota \in L$ , for the extremal label
- a mapping,  $f$ , from the line numbers to transfer functions in  $\mathcal{F}$

$$OUT(n) = \iota_E^n \sqcup \bigsqcup \{IN(n') \mid n \in F. pred(n)\}$$

$$\text{where } \begin{cases} \iota, & \text{if } l \in E \\ \perp, & \text{if } l \notin E \end{cases}$$

$$IN(n) = f_n(OUT(n))$$

# ANALYSIS INSTANCES

| General       | Available Expr                                                                    | Reaching Def               | Very Busy Expr   | Live Variable    |
|---------------|-----------------------------------------------------------------------------------|----------------------------|------------------|------------------|
| $L$           | $\wp(AExp)$                                                                       | $\wp(Var \times Line)$     | $\wp(AExp)$      | $\wp(Var)$       |
| $\sqsubseteq$ | $\supseteq$                                                                       | $\subseteq$                | $\supseteq$      | $\subseteq$      |
| $\sqcup$      | $\cap$                                                                            | $\cup$                     | $\cap$           | $\cup$           |
| $\perp$       | $AExp$                                                                            | $\{\}$                     | $AExp$           | $\{\}$           |
| $\iota$       | $\{\}$                                                                            | $\{x@? \mid x \in FV(F)\}$ | $\{\}$           | $\{\}$           |
| $E$           | $\{init(CFG)\}$                                                                   | $\{init(CFG)\}$            | $\{final(CFG)\}$ | $\{final(CFG)\}$ |
| $F$           | $CFG$                                                                             | $CFG$                      | $CFG^{Reverse}$  | $CFG^{Reverse}$  |
| $\mathcal{F}$ | $\{f: L \rightarrow L \mid \exists l_k, l_g: f(l) = (l \setminus l_k) \cup l_g\}$ |                            |                  |                  |
| $f_l$         | $f_n(l) = (l \setminus kill_n) \cup gen_n$                                        |                            |                  |                  |



# NEXT TIME

Working with Wala frameworks